Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Ingineria Software și Automatică
Program: Software Engineering

Report on laboratory works no. 7 and 8

Course: Operating Systems

Topic: Processes, Redirection and Pipes in Linux and in xv6

Track A (Linux) and Track B (xv6)

Done by:
st. gr. FAF-241,
Serghei Barladean

Checked by:
Patricia Reitman

Chișinău – 2026

Contents

Purpose of the Work....3
Laboratory Work 7, Part 1. fork + exec + wait....3
7.1.1. Work Done....3
7.1.2. Observations....5
Laboratory Work 7, Part 2. A Zombie, on Purpose....5
7.2.1. Work Done....5
7.2.2. Observations....6
Laboratory Work 7, Part 3. Redirection: ls -l > out.txt....6
7.3.1. Work Done....6
7.3.2. Observations....7
Laboratory Work 7, Part 4. A Pipe: ls /etc | wc -l....7
7.4.1. Work Done....7
7.4.2. Optional Task: Three Stages....9
7.4.3. Observations....10
Laboratory Work 8, Part 1. run, Inside xv6....10
8.1.1. Work Done....10
8.1.2. Observations....14
Laboratory Work 8, Part 2. pingpong....14
8.2.1. Work Done....14
8.2.2. Observations....15
Conclusions....16

Purpose of the Work

The lectures asked three questions: how a process is born, where `printf` writes, and how `ls | wc -l` works. The purpose of these two laboratory works is to answer them by running the code. In Laboratory Work 7 three small C programs are compiled and run on Linux: `run.c` (the three calls `fork`, `exec` and `wait` that a shell uses for one command), `redirect.c` (the shell command `ls -l > out.txt` done by hand) and `pipeline.c` (the command `ls /etc | wc -l` done by hand with one pipe and two processes). A zombie process is created on purpose, and the pipeline is broken on purpose to see why a pipe needs its unused ends closed.

In Laboratory Work 8 the same calls are used inside xv6, the small operating system of this course: the program `run.c` is written for xv6 and the program `pingpong.c` makes two processes talk through two pipes. The comparison shows that the system calls are the same in a big OS and in a small one.

The work was done on Ubuntu 24.04 (aarch64) in a Multipass virtual machine on macOS. The Linux programs were compiled with `gcc -Wall -Wextra` in `~/os-lab7`. xv6 (RISC-V) was built in `~/xv6-riscv` and run in the QEMU emulator; both new programs were added to the `UPROGS` list of the Makefile, and every xv6 session started from a freshly built disk image. The figures are taken from the recorded terminal sessions. Laboratory Works 7 and 8 are described in one report, as the handouts allow.

Laboratory Work 7, Part 1. fork + exec + wait

7.1.1. Work Done

The program `run.c` does what the shell does for the command `ls -l`: it copies itself with `fork`, the copy becomes `ls` with `execlp`, and the original waits for it with `waitpid`. The source is exactly the one from the handout:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command:   ls -l   */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();                                 /* step 1: copy myself */
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {                                     /* only the child runs this */
        printf("child: my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);         /* step 2: become ls */
        perror("exec");                                 /* runs only if exec failed */
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);                           /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

The program was compiled and run:

```
gcc -Wall -Wextra -o run run.c && ./run
```

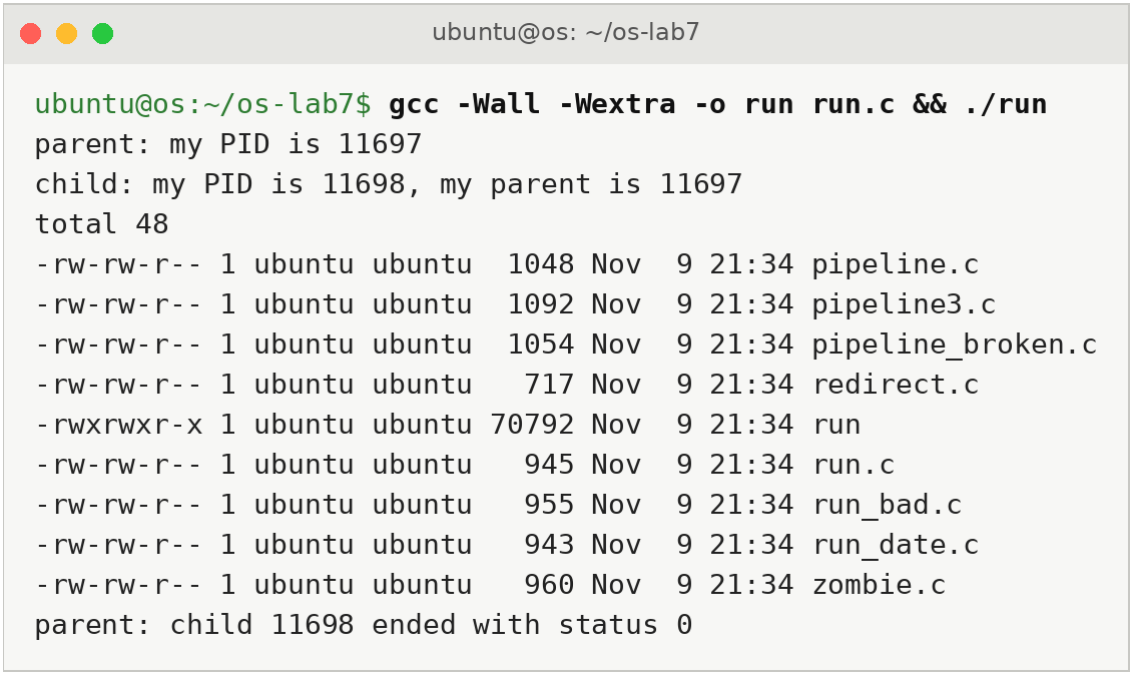


Figure 7.1 – run.c with ls: the parent, the child and the listing

Then the `execlp` line was replaced, first with `date` (the difference was made with `diff`), and then with a program that does not exist:

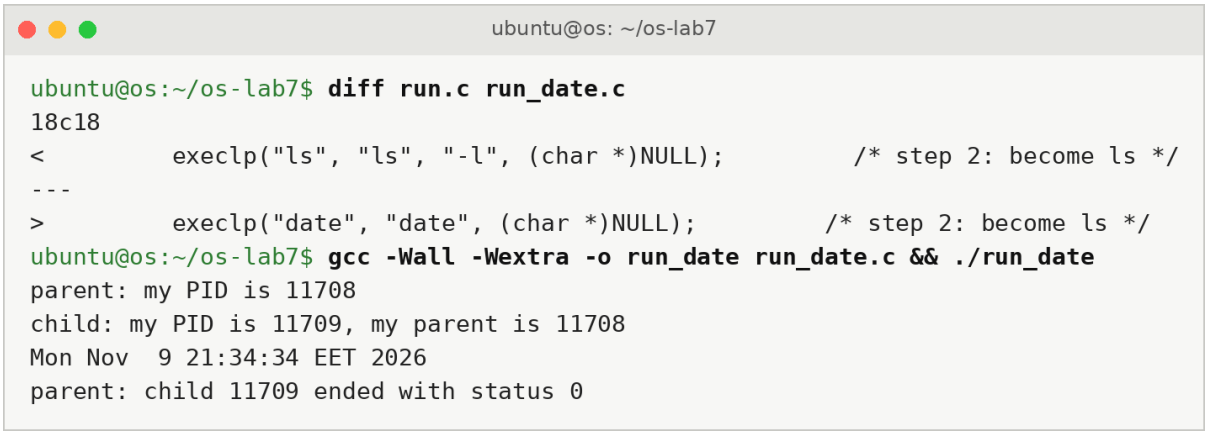


Figure 7.2 – The child becomes date



Figure 7.3 – The child tries to become nosuchprog

7.1.2. Observations

(1) After a successful `execlp` the child process does not have the code of `run` any more: the kernel replaced it with the code of the new program, so the next line `perror("exec")` is never reached. `execlp` returns only if it fails, with the value −1. This happened only with `nosuchprog`: there is no such file, so the old code goes on, `perror` prints `exec: No such file or directory`, and `exit(127)` follows (Figure 7.3).

(2) The child prints as its parent the number 11697 in Figure 7.1, which is the PID printed in the first line by the parent (`parent: my PID is 11697`). So the number is the PID of the `run` process itself, which created the child with `fork`. The same holds in Figures 7.2 and 7.3 (11708 and 11719).

(3) With `ls` (and with `date`) the parent reads status 0, because the program ended without an error. With `nosuchprog` it reads 127, the number given to `exit` after the failed `exec`; 127 is the usual shell code for command not found.

Laboratory Work 7, Part 2. A Zombie, on Purpose

7.2.1. Work Done

A zombie is a child that has ended but was not waited for. The file `zombie.c` is `run.c` with one new line, `sleep(20);`, before `waitpid`:

```
23a24
>     sleep(20);
```

The program was started in the background, and the process table was shown two seconds later, while the parent was asleep:

```
ubuntu@os: ~/os-lab7

ubuntu@os:~/os-lab7$ gcc -Wall -Wextra -o zombie zombie.c
ubuntu@os:~/os-lab7$ ./zombie > /dev/null &
[1] 11732
ubuntu@os:~/os-lab7$ sleep 2; ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
  11680   11678 Ss   bash --noprofile --rcfile /home/ubuntu/os-lab7/.rc -i
  11732   11680 S    ./zombie
  11733   11732 Z    [ls] <defunct>
  11737   11680 R+   ps -o pid,ppid,stat,cmd
```

Figure 7.4 – The finished child [ls] is a zombie while the parent sleeps

After more than 20 seconds `ps` was run again:

```
ubuntu@os: ~/os-lab7

ubuntu@os:~/os-lab7$ ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
  11680   11678 Ss   bash --noprofile --rcfile /home/ubuntu/os-lab7/.rc -i
  11744   11680 R+   ps -o pid,ppid,stat,cmd
[1]+  Done                    ./zombie > /dev/null
```

Figure 7.5 – The process table after the parent has called waitpid

7.2.2. Observations

(1) `ls` has the letter `Z` in the STAT column, which means zombie (the `cmd` column shows `[ls] <defunct>`). The process has already ended and given back its memory, but its entry is kept in the process table together with the exit status, until the parent asks for it. Its parent is `./zombie` (PPID 11732 in Figure 7.4), which is asleep (`S`) at that moment.

(2) No, the line is gone in Figure 7.5; the program `zombie` itself has ended too (`Done ./zombie`). The line that removed the zombie is `waitpid(pid, &status, 0);` in `zombie.c`: when the parent wakes up after `sleep(20)` it calls `waitpid`, the kernel gives it the status of the child and deletes the entry from the table.

Laboratory Work 7, Part 3. Redirection: ls -l > out.txt

7.3.1. Work Done

The program `redirect.c` does what the shell does for `ls -l > out.txt`: in the child it closes descriptor 1, opens the file (which gets the smallest free number) and then runs `ls`:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls -l > out.txt   */
int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        close(1);                                       /* descriptor 1 is now free */
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);         /* ls writes to 1, as always */
        perror("exec");
        exit(127);
    }
    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

It was compiled and run; then the first lines of `out.txt` and its size were shown:

```
gcc -Wall -Wextra -o redirect redirect.c && ./redirect
head -3 out.txt
ls -l out.txt
```

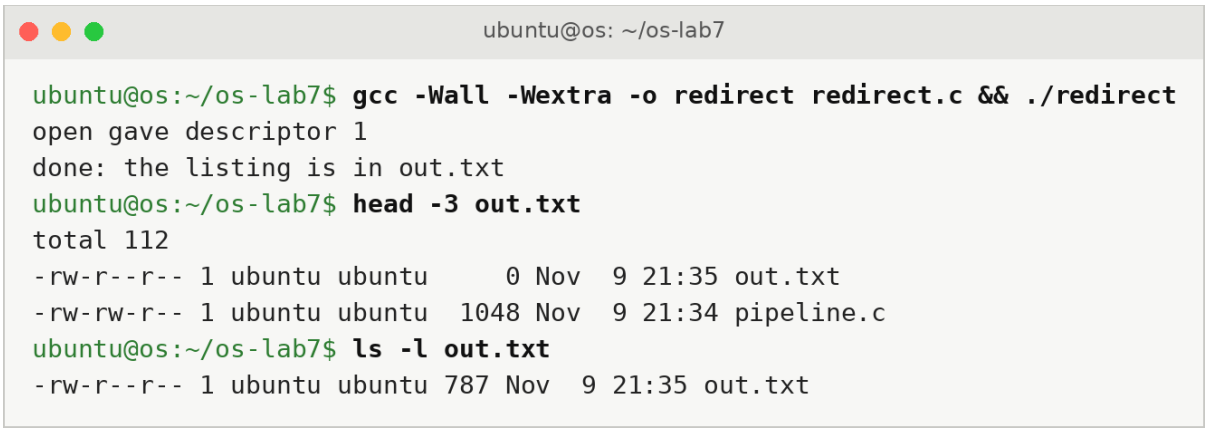


Figure 7.6 – redirect.c: the listing goes into out.txt

7.3.2. Observations

(1) `open` gave descriptor 1 (the message `open gave descriptor 1` in Figure 7.6). `open` always returns the smallest free descriptor number; descriptors 0 and 2 were taken (standard input and standard error), and `close(1)` had just freed number 1.

(2) The program `ls` was not changed; it writes to descriptor 1, as always. But the descriptor table of the child is kept across `exec`, and in this child number 1 does not point to the terminal any more, but to `out.txt`. So the listing is written into the file, and only the messages sent to descriptor 2 (`open gave descriptor 1`) and the line of the parent appear on the screen. The first line of the file `total 112` and the entry `out.txt` with size 0 are a good sign of this: `ls` read the directory when the file had just been created and was still empty (later `ls -l out.txt` shows 787 bytes).

(3) To redirect the errors like `2> err.txt`, descriptor 2 must be closed before `open`, so that the new file gets number 2.

Laboratory Work 7, Part 4. A Pipe: ls /etc | wc -l

7.4.1. Work Done

The program `pipeline.c` builds the pipeline by hand: one `pipe`, two `fork`. In the process with `ls` descriptor 1 becomes the writing end `p[1]`, and in the process with `wc` descriptor 0 becomes the reading end `p[0]`; `dup` copies a descriptor into the smallest free number:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls /etc | wc -l   */
int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }

    if (fork() == 0) {                                  /* ls: its 1 becomes p[1] */
        close(1);
        dup(p[1]);                                      /* smallest free number: 1 */
        close(p[0]);
        close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
```

```
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) {                                  /* wc: its 0 becomes p[0] */
        close(0);
        dup(p[0]);                                      /* smallest free number: 0 */
        close(p[0]);
        close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }

    close(p[0]);                                        /* the parent uses neither end */
    close(p[1]);
    wait(NULL);
    wait(NULL);
    return 0;
}
```

The program was compiled and run, and the result was compared with the real shell:

```
gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
ls /etc | wc -l
```

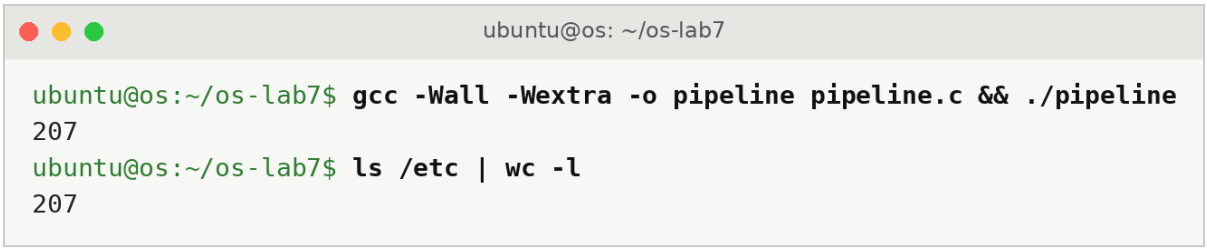


Figure 7.7 – pipeline.c and the shell give the same number

Then the pipeline was broken on purpose: in the parent the line `close(p[1]);` was turned into a comment (the file `pipeline_broken.c`). The program does not end, so it was started in the background; the process table was shown, then the parent was stopped with `kill` and the table was shown again:

```
ubuntu@os: ~/os-lab7
ubuntu@os:~/os-lab7$ diff pipeline.c pipeline_broken.c
33c33
<     close(p[1]);
---
>     /* close(p[1]); */
ubuntu@os:~/os-lab7$ gcc -Wall -Wextra -o pipeline_broken pipeline_broken.c
ubuntu@os:~/os-lab7$ ./pipeline_broken &
[1] 11788
ubuntu@os:~/os-lab7$ sleep 2; ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
  11680   11678 Ss   bash --noprofile --rcfile /home/ubuntu/os-lab7/.rc -i
  11788   11680 S    ./pipeline_broken
  11790   11788 S    wc -l
  11794   11680 R+   ps -o pid,ppid,stat,cmd
ubuntu@os:~/os-lab7$ kill %1
ubuntu@os:~/os-lab7$ sleep 1; ps -o pid,ppid,stat,cmd
[1]+  Terminated              ./pipeline_broken
    PID    PPID STAT CMD
  11680   11678 Ss   bash --noprofile --rcfile /home/ubuntu/os-lab7/.rc -i
  11799   11680 R+   ps -o pid,ppid,stat,cmd
```

Figure 7.8 – The broken pipeline: wc never ends

7.4.2. Optional Task: Three Stages

The program `pipeline3.c` does the command `ls /usr/bin | grep x | wc -l`: two pipes, three children. Every child closes all four pipe ends after its `dup`, and so does the parent:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls /usr/bin | grep x | wc -l   */
int main(void)
{
    int a[2], b[2];
    if (pipe(a) < 0 || pipe(b) < 0) { perror("pipe"); return 1; }

    if (fork() == 0) {
        close(1);
        dup(a[1]);
        close(a[0]); close(a[1]); close(b[0]); close(b[1]);
        execlp("ls", "ls", "/usr/bin", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) {
```

```
        close(0);
        dup(a[0]);
        close(1);
        dup(b[1]);
        close(a[0]); close(a[1]); close(b[0]); close(b[1]);
        execlp("grep", "grep", "x", (char *)NULL);
        perror("exec grep");
        exit(127);
    }

    if (fork() == 0) {
        close(0);
        dup(b[0]);
        close(a[0]); close(a[1]); close(b[0]); close(b[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }

    close(a[0]); close(a[1]); close(b[0]); close(b[1]);
    wait(NULL);
    wait(NULL);
    wait(NULL);
    return 0;
}
```

```
ubuntu@os: ~/os-lab7

ubuntu@os:~/os-lab7$ gcc -Wall -Wextra -o pipeline3 pipeline3.c && ./pipeline3
156
ubuntu@os:~/os-lab7$ ls /usr/bin | grep x | wc -l
156
```

Figure 7.9 – pipeline3.c and the shell give the same number

7.4.3. Observations

(1) Yes. Both `./pipeline` and `ls /etc | wc -l` printed 207 (Figure 7.7). The optional program `pipeline3` printed 156, the same as the shell command (Figure 7.9).

(2) After the change the process `wc` never ends: Figure 7.8 shows `wc -l` in state `S` (sleeping) with the parent `pipeline_broken` as its parent. `ls` is not in the table any more: it wrote its names, ended and was collected by the first `wait`.

(3) `wc` is waiting for the end of its input: its `read` on the pipe returns 0 (end of file) only when every copy of the writing end `p[1]` is closed, and the parent still holds one copy open, so `wc` waits for more data that never comes. This also explains the last lines of Figure 7.8: after `kill` the parent died, the kernel closed its descriptors, `wc` got the end of file and ended too.

Laboratory Work 8, Part 1. run, Inside xv6

8.1.1. Work Done

The same three steps as in `run.c` of Laboratory Work 7 were written for xv6. There is no glibc here, only the system calls of the kernel that was compiled. The child becomes `echo`. The program `user/run.c`:

```
#include "kernel/types.h"
```

```c
#include "user/user.h"

// the three steps of the shell, inside xv6: echo hello from the child
int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());

  int pid = fork();                              // step 1: copy myself
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }

  if(pid == 0){                          // only the child runs this
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args);                  // step 2: become echo
    printf("exec failed\n");             // runs only if exec failed
    exit(1);
  }

  int status;
  wait(&status);                         // step 3: wait for the child
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

The line `$U/_run\` was added to the `UPROGS` list of the Makefile (the same for `pingpong`, Figure 8.4), then xv6 was started with `make qemu` and the program was run twice:

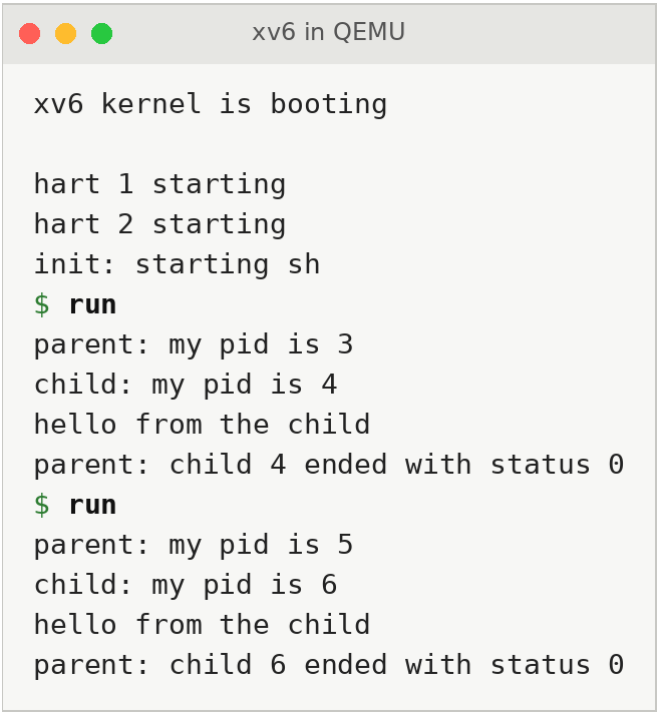


Figure 8.1 – run in xv6: the child becomes echo

```
ubuntu@os: ~/xv6-riscv

ubuntu@os:~/xv6-riscv$ grep -n "_run\|_pingpong" Makefile
135:    $U/_pingpong\
136:    $U/_run\
```

Figure 8.4 – The new lines in the UPROGS list of the Makefile

Then the child was made to run `ls`: two lines were changed (the difference is shown by `diff`) and xv6 was built and started again:

```
18,19c18,19
<     char *args[] = { "echo", "hello", "from", "the", "child", 0 };
<     exec("echo", args);                 // step 2: become echo
---
>     char *args[] = { "ls", 0 };
>     exec("ls", args);                   // step 2: become ls
```

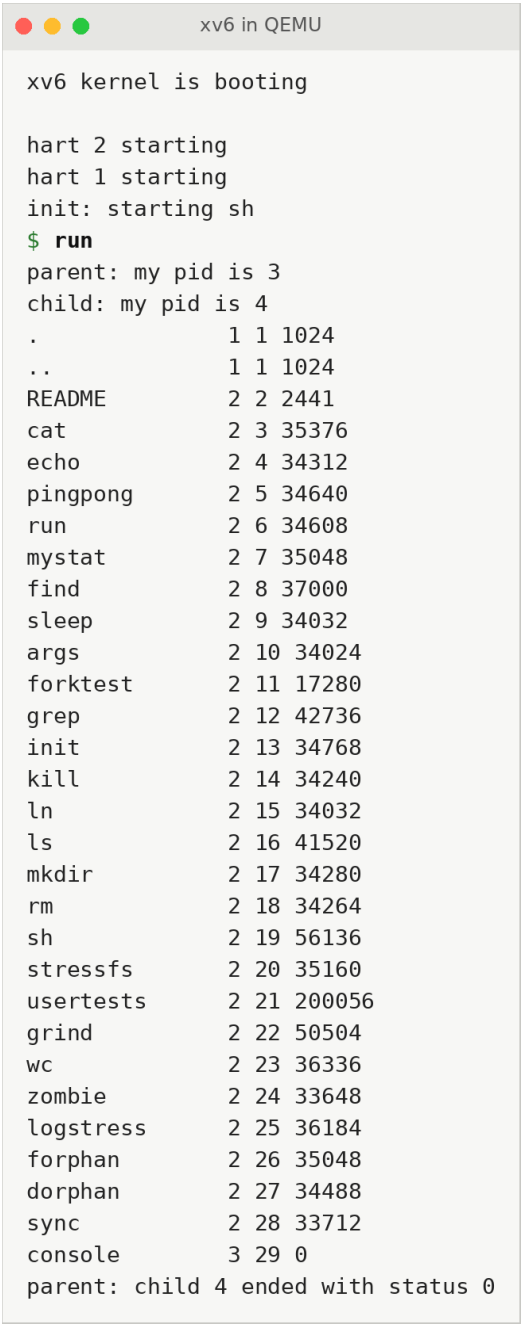


Figure 8.2 – run in xv6: the child becomes ls

Last, a program that does not exist was used in `exec`:

```
19c19
<     exec("echo", args);                // step 2: become echo
---
>     exec("nosuchprog", args);          // step 2: become echo
```

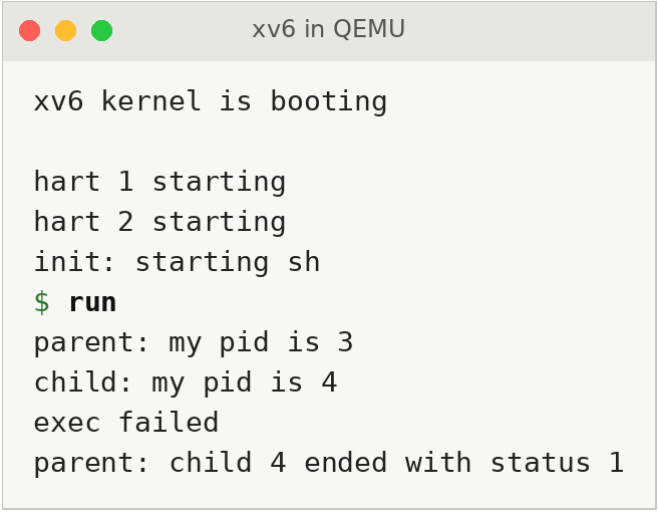


Figure 8.3 – run in xv6: exec of a program that does not exist

8.1.2. Observations

(1) In xv6 `run.c` uses `fork`, `exec` and `wait`. In Linux `run.c` used `fork`, `execlp` and `waitpid`. They are the same three steps with the same meaning: `fork` makes a copy of the process, `exec` replaces the code of the copy with a new program, and `wait` gives the parent the status of the child. The differences are small: in Linux the library function `execlp` looks for the program in the `PATH` and takes the arguments one by one, in xv6 `exec` takes a path and an array of arguments that ends with 0; `waitpid` names the child and gives the status in an encoded form (the macro `WEXITSTATUS` is needed), while the xv6 `wait` takes any child and puts the exit number directly into `status`.

(2) The text `hello from the child` is printed by `echo`, not by the code of `run`. The program `run` prints only the lines with `parent:` and `child:`. After `exec("echo", args)` the child (pid 4 in Figure 8.1) is the program `echo` with the arguments `hello from the child`; the old code of `run` is gone from this process, and `echo` writes to descriptor 1. With `ls` the same child printed the directory listing (Figure 8.2).

(3) The status is the number given to `exit`. If `exec` fails, the code of `run` goes on: it prints `exec failed` and calls `exit(1)`, so the parent reads 1 (Figure 8.3). In the Linux program the line after the failed `execlp` was `exit(127)`, so the status was 127. The authors only chose different numbers; 127 is the convention of the shell for command not found.

Laboratory Work 8, Part 2. pingpong

8.2.1. Work Done

The program `user/pingpong.c` sends one byte from the parent to the child through the first pipe `p2c`. The child prints its own pid with `received ping`, writes the byte back through the second pipe `c2p` and exits. The parent reads it, prints `received pong` and waits for the child. The four places for `read` and `write` from the handout were completed (the parent sets the byte to `x` before sending it):

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];     // parent-to-child, child-to-parent
```

```
  char b;

  pipe(p2c);
  pipe(c2p);
  if(fork() == 0){
    close(p2c[1]); close(c2p[0]);       // the ends the child does not use
    read(p2c[0], &b, 1);
    printf("%d: received ping\n", getpid());
    write(c2p[1], &b, 1);
    exit(0);
  }
  close(p2c[0]); close(c2p[1]);         // the ends the parent does not use
  b = 'x';
  write(p2c[1], &b, 1);
  read(c2p[0], &b, 1);
  printf("%d: received pong\n", getpid());
  wait(0);
  exit(0);
}
```

The line `$U/_pingpong\` is in `UPROGS` (Figure 8.4). The program was run three times in xv6, the third time with the output through a pipe:

```
pingpong
pingpong
pingpong | cat
```



Figure 8.5 – pingpong in xv6

8.2.2. Observations

(1) A pipe is a one-way channel: the bytes go in at the writing end and come out in the same order at the reading end. With a single pipe for both directions the parent would write the byte and then read from the same pipe, and it could take back its own byte before the child has seen it (or the child would get the byte, write it back and read it again itself). Two pipes, `p2c` for the parent to the child and `c2p` for the child to the parent, keep the two directions apart. That is also why every process closes the two ends it does not use.

(2) The call `read(c2p[0], &b, 1)` in the parent makes it wait. A `read` on an empty pipe whose writing end is still open puts the process to sleep until a byte arrives, and the child writes the byte only after it has received the ping. The same blocking `read(p2c[0], &b, 1)` makes the child wait for the parent, so `ping` is always printed first, by the child, and `pong` second, by the parent (Figure 8.5). The `wait(0)` at the end only collects the finished child.

Conclusions

In Laboratory Work 7 the work of the shell was done by hand on Linux. A process is created by `fork` and becomes another program with `exec`, and the parent collects its exit status with `wait`. If `exec` does not succeed, the old code goes on, which gives the status 127 for a program that does not exist. A child that has ended and was not waited for stays in the process table as a zombie until the parent calls `waitpid`. Redirection needs no change in the program that is started: the shell closes a descriptor, `open` takes the smallest free number, and the descriptor stays open across `exec`. A pipe works the same way with `dup`; the experiment with the missing `close(p[1])` showed that the reader of a pipe gets the end of file only when all copies of the writing end are closed.

In Laboratory Work 8 the same three calls were used in xv6 and gave the same behaviour: the child became `echo` or `ls`, and a failed `exec` gave the status chosen in the program. The program `pingpong` showed how two processes exchange a byte through two pipes, and that a blocking `read` is enough to synchronise them without any other waiting code. The programs of both works were compiled and tested, `run.c` and `pingpong.c` were added to the `UPROGS` list of the Makefile, and the code is pushed to the Git repository together with this report.